

Jazz Learning App

Proof-of-Concept Design Document — JUCE Standalone Application

1. Overview

A standalone, cross-platform jazz education application built on the JUCE framework, targeting desktop (macOS/Windows) and mobile (iOS/Android) from a shared codebase. The app helps jazz musicians build and reharmonize chord progressions, understand which scales fit each chord, and get real-time feedback on the voicings they play. The POC scope covers two core modules: the Reharmonization Assistant and the Real-Time Chord/Voicing Analyzer, sharing a single responsive UI layer and a UI-agnostic music theory/analysis engine.

2. Goals for this POC

- Prove out the core theory engine: chord parsing, scale-suggestion logic, and voicing analysis.
- Validate a single responsive UI approach across desktop and mobile form factors.
- Support MIDI keyboard input and on-screen keyboard input (multi-touch chord entry) as the two input methods — no audio/pitch-detection input in this phase.
- Establish the module boundary between UI and core logic early, so the theory engine is portable to a future AUV3/VST3 plugin target without a rewrite.

3. Architecture

3.1 High-Level Structure

The project is split into three layers, structured as separate JUCE modules so the theory engine has no dependency on UI or platform code:

Layer	Responsibility	Depends On
Core Engine	Chord parsing, reharmonization logic, scale suggestion, voicing analysis	Nothing (Platform-specific JUCE GUI classes.)
Shared UI Component Library	Chart view, on-screen keyboard widget, scale-suggestion panel, Chord Engine (back panel)	Core Engine (back panel) — each with size-class
Platform Shell	MIDI I/O per platform, file import/export (iReal Pro, MusicXML), App lifecycle	App-specific APIs

3.2 UI Strategy: Single Responsive UI

One shared UI component library is used across desktop and mobile rather than separate platform-specific UIs. This is a deliberate scope decision for a solo/small-team POC: maintaining two UI codebases roughly doubles UI development and testing effort, and the app's interaction complexity (chart view, tappable measures, keyboard input, feedback panels) does not require fundamentally different interaction paradigms between mouse and touch.

Components adapt via **size classes** (compact / regular / expanded) rather than hardcoded pixel layouts, using JUCE's FlexBox/Grid layout system:

- **Layout breakpoints:** multi-pane views (chart + analyzer side-by-side) collapse into tabbed/stacked views below a width threshold.
- **Interaction affordances:** the reharmon assistant's scale dropdown becomes a bottom sheet/modal on touch, a standard dropdown on desktop — same underlying component, different presentation.
- **On-screen keyboard:** same widget everywhere, but default visible octave range and multi-touch handling are tuned per screen size.
- **Popups/modals:** desktop supports hover states and smaller floating popups; touch uses larger tap targets and full-sheet modals.

Future consideration (post-POC): a denser, DAW-style power-user layout for desktop, treated as an additional size class rather than a second UI, unless divergence becomes extreme.

4. Feature Modules

4.1 Reharmonization Assistant

Core functionality:

- Create a chord progression manually, or import from an iReal Pro chart (MusicXML/MuseScore import as a stretch goal if feasible).
- Tapping/clicking a measure opens a panel showing the suggested scale for improvising over that chord, plus a dropdown (desktop) or bottom sheet (mobile) listing all other valid scale options for that chord.

Additional functionality to consider:

- **Voice-leading visualizer:** shows how guide tones (3rds/7ths) move between measures after a reharmon substitution, so users can evaluate whether new changes flow smoothly.
- **Style presets for suggestions:** bias reharmon suggestions toward a chosen vocabulary (bebop, modal/Bill Evans-style, quartal, Brazilian) instead of one generic engine.
- **Difficulty tagging:** flag substitutions as "safe" (diatonic, common) vs. "advanced" (tritone subs, altered dominants) so learners can progress gradually.
- **Export/print chart:** export a finalized reharmon back to iReal Pro format or as a PDF lead sheet for gig/rehearsal use.

4.2 Real-Time Chord/Voicing Analyzer

Core functionality:

- Hooks up to a loaded chord chart or manually entered progression.
- User plays a chord on a MIDI keyboard or the on-screen keyboard; the app analyzes the voicing against the expected chord symbol — accounting for voicing type (root-position solo piano, two-handed voicing with no root, left-hand-only voicing with no root, etc.) — and gives recommendations for improving it.

Additional functionality to consider:

- **Solo/improv feedback layer:** listens to a soloed line over the progression and flags whether note choices align with the suggested scale/chord tones (framed as "outside the target scale," not "wrong").

- **Voicing library builder:** save confirmed good voicings into a personal reference library organized by chord type/inversion, building a growing practice resource over time.
- **Voicing "sentence starters":** for chord types the user struggles with, show 2–3 idiomatic voicing options as scaffolding before they attempt it themselves.

4.3 Supporting Modules (Round Out the Practice Loop)

- **Ear training tied to the loaded chart:** quiz the user on identifying chord quality/extensions from measures in their own chart, reinforcing theory in context rather than with generic drills.
- **Metronome / practice-loop mode:** loop a difficult section (e.g., a bridge or a tricky ii–V–I) at adjustable tempo, integrated with the analyzer for real-time feedback while looping.
- **Progress tracking:** simple stats over time — voicing accuracy %, weakest chord types, reharmon styles explored — so practice has visible direction.

5. Input & Platform Considerations

5.1 Input Methods (POC Scope)

- **MIDI keyboard** — hardware input via USB or Bluetooth MIDI.
- **On-screen keyboard** — first-class input method, not a fallback, since many mobile users will never own a MIDI keyboard.

Audio/pitch-detection input (mic or pickup) is explicitly out of scope for the POC — it's a significantly larger DSP lift and is treated as a future stretch goal.

5.2 MIDI on Mobile

- **iOS:** Bluetooth MIDI keyboards work out of the box via JUCE's MIDI API; USB MIDI keyboards work through a Lightning/USB-C camera adapter.
- **Android:** Bluetooth MIDI supported since Android 6; USB MIDI works via OTG adapters. JUCE's Android MIDI handling is less mature than iOS — budget extra testing time here as the likely source of platform-specific bugs.
- MIDI input should be abstracted behind a single interface so the reharmonization/analysis logic never needs to know whether a note came from hardware MIDI or the on-screen keyboard.

5.3 On-Screen Keyboard Design

- Must support multi-touch chord entry — multiple simultaneous touches register as a single "voicing" event, matching how the analyzer expects to receive a full chord rather than a stream of individual notes.
- Full 88-key layout is impractical on a phone; use an octave-shiftable 1–2 octave keyboard, similar to GarageBand/Cubasis on iOS.

5.4 App Target Type

Built as a JUCE "GUI Application" (standalone) target rather than an audio plugin client — compiling to iOS/Android/desktop from one codebase via Projucer. If a future "check your voicing while playing in your DAW" mode is desired, that becomes a separate AUv3/VST3 target sharing the same Core Engine module — supported by keeping the engine UI-agnostic from day one.

6. Open Questions / Future Decisions

- Import scope: is iReal Pro import sufficient for the POC, or is MusicXML/MuseScore import needed on day one?
 - How much of the reharmonization suggestion engine should be rule-based (music-theory heuristics) vs. data/ML-informed, at least for the POC?
 - Scope and timing of the solo/improv feedback layer relative to the two core modules — full feature or post-POC addition?
 - Whether a future DAW-style dense desktop layout is worth designing for early, or deferred until the responsive system is proven.
-

Draft design document — scope and priorities subject to change as the POC develops.